Rilevamento e mitigazione DoS/DDoS con protezione da ARP spoofing in una rete SDN con Ryu

Network and Cloud Infrastructures – Prof. G. Ventre – A.A. 2025/2026

Gianmarco La Salvia (matricola DE9000140)

Repository GitHub: https://github.com/giovannistanco/NCIS_unina/tree/main/2026/La_Salvia_Gianmarco

1 Introduzione e obiettivi

Nelle reti SDN il controller vede tutto il traffico e può reagire con regole OpenFlow, ed è quindi il punto naturale in cui rilevare e mitigare attacchi Denial of Service (DoS/DDoS). Le contromisure semplici, come bloccare l'intera porta più carica, soffrono però di *over-blocking*: i flussi legittimi che condividono porta o uplink con l'attacco vengono colpiti insieme ad esso. Il progetto parte dai limiti dell'approccio classico (Tabella 1) e propone un sistema che:

- emula una rete SDN (Mininet, Open vSwitch, Ryu, OpenFlow 1.3) con **due attaccanti su switch diversi** e misura il degrado dei client legittimi;
- rileva gli attacchi UDP con una **soglia adattiva** (EWMA) calcolata *per porta di accesso*, quindi robusta al cambio di IP, MAC o destinazione;
- mitiga in modo **granulare a due livelli** (blocco IP, poi quarantena della porta) con backoff esponenziale e decadimento della penalità;
- protegge il controllo della rete: ARP Guard (e IP Guard) contro lo spoofing e **REST API amministrativa autenticata**;
- confronta quantitativamente protezione OFF e ON.

Limite dell'approccio classico	**Soluzione adottata**
Over-blocking (blocco di tutta la porta)	Blocco granulare `porta+IP`; quarantena solo sulla porta di accesso del recidivo, mai sugli uplink
Soglia statica	EWMA per porta con tetto assoluto P_{max} contro l'avvelenamento della baseline
Decisioni solo nel controller	REST API autenticata con blocklist condivisa e modificabile dall'amministratore
Poca modularità	Moduli logici separati (monitor, detector, enforcer, guardie, REST) con lock sullo stato condiviso
Un solo attaccante, topologia fissa	Due attaccanti su switch diversi, topologia a 4 switch ad albero, evasione con cambio di IP testata
Sblocco troppo presto o troppo tardi	Backoff esponenziale con tetto e decadimento della penalità dopo un periodo di quiete

Tabella 1: Limiti dell'approccio classico e soluzioni adottate.

2 Architettura della rete

La topologia (Figura 1) è un albero senza cicli (quindi il forwarding L2 appreso è sufficiente). Gli attaccanti h2 e h4 sono su switch di accesso diversi e, accanto a ciascuno, c'è un client legittimo (h1, h3): così si verifica l'assenza di danni collaterali. Il collo di bottiglia è il link s3–s4 da 20 Mbit/s.

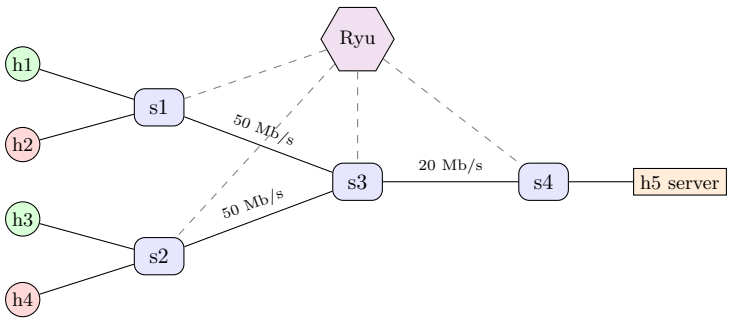


Figura 1: Topologia. Verde: client legittimi (h1, h3). Rosso: attaccanti (h2, h4). Link host–switch a 100 Mbit/s, s4–h5 a 100 Mbit/s (`TCLink`). Tratteggiato: OpenFlow 1.3.

Traffico. h1 e h3: `ping`, `curl` e `iperf` TCP verso h5. h2 e h4: flood UDP (`iperf -u -b 100M`). h5: server HTTP (porta 80) e server `iperf` TCP/UDP (porta 5001). Il controller non vede mai i client come "attaccanti" a priori: la classificazione avviene solo dal comportamento.

3 Progettazione del controller

Il controller è l'app Ryu `SdnGuard` (`controller.py`); i moduli logici sono in Tabella 2. La protezione si attiva con la variabile `PROTECTION=1`; con `PROTECTION=0` il controller resta un semplice learning switch (scenario "senza protezione").

Modulo	Funzione
Forwarding	Learning switch L2. Flow a priorità 10 con match su `in_port, ipv4_src, ipv4_dst, ip_proto`, *senza porte L4*: un flood con porte variabili resta in un solo flow e i contatori aggregano il traffico
Monitor	Ogni Δt richiede le `FlowStats`; calcola i pkt/s come (nuovo − precedente)/Δt gestendo i contatori azzerati dai flow scaduti
Detector	Per ogni porta di accesso somma i pkt/s UDP di tutti i flussi e li confronta con la soglia adattiva
Enforcer	Installa drop a priorità 100 sullo switch di ingresso: prima per `porta+IP`, alla recidiva sull'intera porta di accesso
ARP Guard	Trust On First Use (TOFU): scarta un ARP se l'IP è già legato a un altro MAC o se MAC ARP e MAC Ethernet differiscono
IP Guard	Scarta (con drop in hardware) i pacchetti IP il cui IP sorgente è legato a un altro MAC
REST API	Blocco manuale per IP, lista della blocklist; autenticata (Sez. 3.3)

Tabella 2: Moduli del controller.

3.1 Rilevamento adattivo per porta

Per ogni porta di accesso p si calcola il rate UDP aggregato x_p (pkt/s). Media e deviazione si aggiornano con una EWMA e la soglia ha un tetto assoluto:

$$\mu \leftarrow \mu + \alpha(x-\mu), \quad \sigma \leftarrow (1-\alpha)\sigma + \alpha|x-\mu|, \quad T = \min\big(P_{max},\ \max(P_{min},\ \mu + k\sigma)\big).$$

L'allarme scatta se $x_p > T$ e in tal caso la baseline *non* viene aggiornata. Il tetto P_{max} impedisce che un attaccante che parte subito a pieno ritmo "insegni" alla baseline che il suo rate è normale. Aggregare per porta, e non per flusso o per IP, rende il rilevamento indipendente da IP, MAC e destinazione dei flussi (per costruzione; nei test è stato verificato il cambio di IP).

3.2 Mitigazione a due livelli, backoff e decadimento

Alla n-esima violazione della stessa porta: $n = 1$ blocca `porta+IP sorgente` (granulare); $n \geq 2$ (recidiva, anche con IP o MAC cambiati) mette in *quarantena* la porta di accesso. Il ban dura $t_n = \min(t_{max},\ t_0\, 2^{\,n-1})$. Dopo la fine del ban, ogni T_d secondi di quiete la penalità scende di 1 (calcolato alla violazione successiva), così un host che smette di attaccare viene perdonato. La quarantena riguarda solo la porta dell'attaccante (si assume una porta di accesso per host, come con la *port security*), mai i link tra switch.

Parametro	Valore	Motivazione
Intervallo di polling Δt	2 s	Compromesso fra reattività e carico del controller; dà un tempo di rilevamento nell'ordine di pochi secondi
α, k	1/8, 3	$\alpha = 1/8$ è il peso classico degli stimatori EWMA (es. RTT TCP); $k = 3$ per limitare i falsi positivi
P_{min}	300 pkt/s	≈3,5 Mbit/s con datagrammi da 1470 B (circa il 17% del collo di bottiglia da 20 Mbit/s); sopra il traffico UDP legittimo dei test
P_{max}	800 pkt/s	≈9,4 Mbit/s, vicino alla quota equa di due sorgenti sul collo di bottiglia
t_0, t_{max}, T_d	10 s, 120 s, 60 s	Ban iniziale breve, tetto al raddoppio, decadimento dopo un minuto di quiete

Tabella 3: Parametri scelti e motivazione.

3.3 Sicurezza dell'interfaccia di amministrazione

L'API (`POST /firewall/block`, `GET /firewall/blocklist`) permette all'amministratore di contribuire alla blocklist. Misure: token `Bearer` (da `ADMIN_TOKEN`) confrontato in tempo costante; blocco del client dopo 5 tentativi falliti per 60 s (HTTP 429); validazione di IP e durata; IP protetti (il server h5) non bloccabili; ascolto solo su `127.0.0.1`; audit nei log. Limiti: token statico e HTTP senza TLS (accettabile solo in locale).

4 Metodologia sperimentale

Esecuzione. Prima il controller, poi la topologia (dopo ogni riavvio del controller si riavvia anche Mininet con `sudo mn -c`):

```
ADMIN_TOKEN=<segreto> PROTECTION=1 ryu-manager --wsapi-host 127.0.0.1 controller.py   # PROTECTION=0 per lo scenario senza protezione
sudo python3 run_experiments.py --label on --scenario ddos --reps 3
```

Metriche (misurate da h1 verso h5, media ± deviazione standard su 3 ripetizioni): latenza media e jitter (`mdev` di `ping`), perdita, throughput TCP (`iperf -i 1`) e *tempo di rilevamento*, ossia la differenza fra l'inizio del flood (`date`) e il timestamp `[Detector]` nel log. Le metriche quantitative (E1-E4) sono medie sull'intera run di 40 s (8 s prima dell'attacco, 20 s di attacco, 12 s dopo). Protocolli manuali: in E5 si riprende il flood dopo aver cambiato IP; in E6 si invia ping da h1 durante la quarantena di h2; in E7 si forza un cambio IP su h3 e si invia traffico; in E8 si effettuano richieste `curl` (con e senza token).

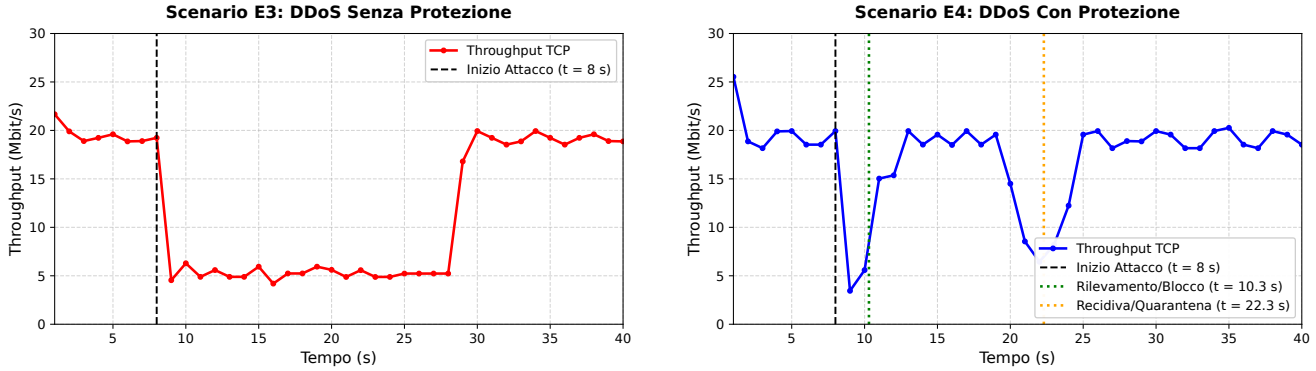
ID	Scenario	Cosa si verifica
E1	Baseline	Prestazioni senza attacco (riferimento)
E2	Attacco singolo (h2), OFF	Degrado di h1 con un solo attaccante
E3	DDoS (h2+h4), OFF	Degrado di h1 con due attaccanti
E4	DDoS (h2+h4), ON	Rilevamento, mitigazione e ripristino di h1
E5	Evasione: cambio IP, ON	1^a violazione IP $\rightarrow$ 2^a: quarantena; backoff e decadimento
E6	Quarantena di h2, ON	h1 (stesso switch) non subisce danni collaterali
E7	Spoofing ARP, ON	h3 rivendica l'IP di h4: l'ARP Guard lo scarta e h3 non comunica
E8	REST senza/con token	Risposte 401, 200 (blocco 429 implementato ma non mostrato)

Tabella 4: Scenari sperimentali.

5 Risultati

Metrica (h1 $\rightarrow$ h5)	E1 Baseline	E2 1 att. OFF	E3 DDoS OFF	E4 DDoS ON
Latenza media (ms)	$0{,}17 \pm 0{,}01$	$20{,}12 \pm 0{,}18$	$34{,}58 \pm 0{,}87$	$9{,}81 \pm 0{,}55$
Jitter, mdev (ms)	$0{,}68 \pm 0{,}03$	$20{,}53 \pm 0{,}23$	$35{,}46 \pm 2{,}07$	$28{,}37 \pm 1{,}66$
Perdita ping (%)	0	0	0	0
Throughput TCP (Mbit/s)	$19{,}10 \pm 0{,}00$	$13{,}00 \pm 1{,}54$	$12{,}17 \pm 0{,}46$	$17{,}10 \pm 0{,}36$
Tempo di rilevamento (s)	–	–	–	$2{,}3 \pm 0{,}3$

Tabella 5: Prestazioni del client legittimo h1 (media $\pm$ dev. std. su 3 ripetizioni). Il flood DDoS non mitigato (E3) riduce il throughput TCP di h1 da 19,10 a 12,17 Mbit/s e innalza la latenza da 0,17 a 34,58 ms; con protezione attiva (E4), il detector interviene in 2,3 s consentendo il ripristino del throughput a 17,10 Mbit/s medi sull'intera run (18,9 Mbit/s a regime).



(a) Senza protezione **(b)** Con protezione

Figura 2: Throughput TCP di h1 durante il DDoS (E3 e E4).

Evasione, backoff e decadimento (E5). Dopo il blocco di h2, l'attaccante cambia IP (da 10.0.0.2 a 10.0.0.99, stesso MAC e stessa porta) e riprende il flood: il detector, che lavora per porta, lo individua comunque (Figura 3(a)).

t (s)	Evento	Azione dell'Enforcer	Violazione #	Ban (s)
18:06:51 (+0 s)	Flood da 10.0.0.2	Blocco `porta+IP`	1	10
18:07:11 (+20 s)	Flood da nuovo IP (stessa porta)	Quarantena porta 2 di s1	2	20
18:09:41 (+170 s)	Nuovo flood dopo $\approx$130 s di quiete	Blocco `porta+IP` (penalità decaduta)	1	10

Tabella 6: Cronologia da log del controller. Alla recidiva sulla stessa porta scatta la quarantena (20 s); trascorsi 130 s di quiete ($> 2 \times T_d$), la penalità è decaduta e la nuova violazione riceve la sanzione base di 10 s.

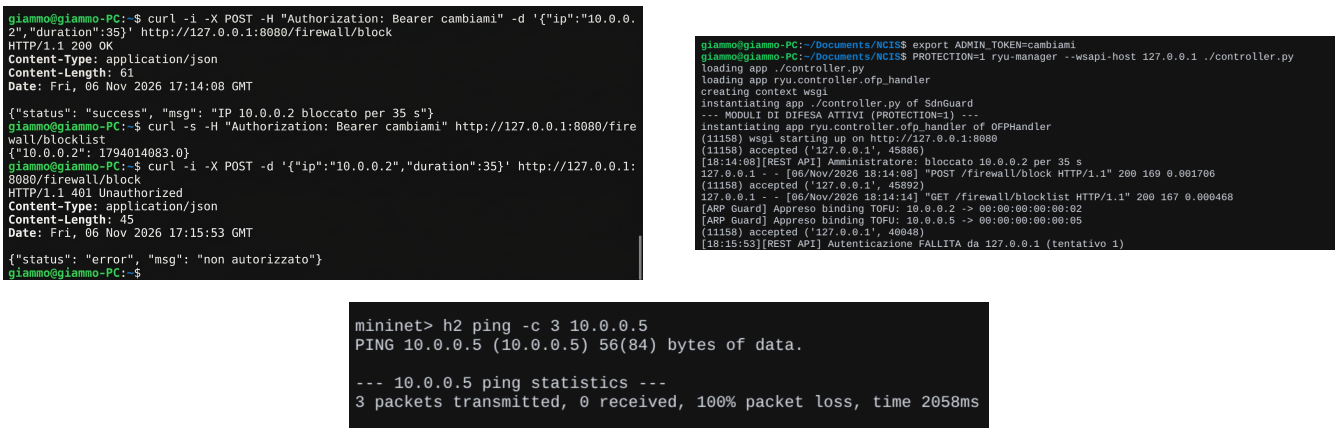
Assenza di over-blocking (E6). Durante la quarantena della porta 2 di s1, h1 (stesso switch) ha 0% di perdita; su 3 pacchetti la latenza media è 3,07 ms (8,6 ms il primo, probabilmente per l'installazione del flow, poi 0,52 e 0,09 ms), contro i 34,58 ms di E3. In Fig. 2(b) il throughput di h1 torna a circa 19 Mbit/s mentre le porte degli attaccanti sono in quarantena. Il test mostra l'assenza di blocco di h1 ma, su 3 pacchetti, non quantifica un eventuale degrado residuo.



(a) Evasione DoS e assenza di over-blocking (E6): escalation a quarantena della porta e ping di h1 con 0% di perdita.



(b) ARP/IP spoofing (E7): rilevamento e drop del furto d'identità di h3 nel log Ryu con esito unreachable in Mininet.



(c) REST API autenticata (E8): chiamate curl (200/401), esecuzione nel log Ryu e ping bloccato di h2 (100% loss).

Figura 3: Evidenze sperimentali (E5, E6, E7, E8).

L'analisi sperimentale evidenzia quattro aspetti determinanti: (i) l'attacco singolo (E2) e il DDoS (E3) degradano entrambi h1: throughput medio $13{,}00 \pm 1{,}54$ e $12{,}17 \pm 0{,}46$ Mbit/s, latenza $20{,}12 \pm 0{,}18$ e $34{,}58 \pm 0{,}87$ ms; il DDoS peggiora soprattutto la latenza; (ii) l'algoritmo adattivo EWMA (E4) rileva e isola l'aggressione in appena $2{,}3 \pm 0{,}3$ s (in linea con $\Delta t = 2$ s), ripristinando il throughput a $17{,}10 \pm 0{,}36$ Mbit/s medi sull'intera run; (iii) la quarantena della porta contrasta l'evasione tramite cambio di IP dalla stessa porta (E5), preservando intatta l'operatività del client legittimo h1 sullo stesso switch (E6, 0% perdite); (iv) l'ARP Guard scarta nel controller gli ARP con cui h3 rivendica l'IP di h4 e h3 non riesce a comunicare (E7, Figura 3(b)); l'IP Guard è implementato ma non valutato sperimentalmente; le API REST autenticate garantiscono una gestione affidabile e protetta delle policy esterne (E8, Figura 3(c)).

6 Limiti

- **TOFU.** L'ARP Guard si fida della *prima* associazione IP–MAC osservata. Se un attaccante invia per primo un ARP per l'IP di un altro host, il binding resta avvelenato e l'host legittimo viene scartato. Abbiamo scelto un binding dinamico invece di MAC fissi nel codice per flessibilità; in uno scenario reale la tabella affidabile si costruisce osservando gli scambi DHCP (*DHCP snooping*, base del Dynamic ARP Inspection). Nei test i binding si imparano all'avvio (ARP di `pingall`), prima di ogni attacco. L'IP Guard si applica solo agli IP con un binding già appreso, e un cambio legittimo di MAC richiede il riavvio del controller.
- **Copertura del detector.** Solo traffico UDP, aggregato per porta: SYN flood o ICMP flood non sono coperti; un attacco *low-rate* sotto P_{min} non viene rilevato; P_{max} è un parametro tarato sulla nostra topologia.
- **Porte condivise.** La quarantena assume una porta di accesso per host; con più host sulla stessa porta colpirebbe anche i vicini.
- **Controller come bersaglio.** Il match senza porte L4 e il drop in hardware limitano i packet-in, ma un attaccante che cambia continuamente IP genera comunque nuovi flow.

- **REST.** Token statico e HTTP in chiaro (solo `localhost`), senza ruoli.

7 Conclusioni

Il sistema rileva il flood UDP in $2{,}3 \pm 0{,}3$ s e lo mitiga sullo switch di ingresso: nel DDoS il throughput medio di h1 sull'intera run passa da $12{,}17 \pm 0{,}46$ a $17{,}10 \pm 0{,}36$ Mbit/s (baseline 19,10). Il secondo calo di Fig. 2(b) ($t \approx 20-24$ s) corrisponde alla scadenza del primo ban da 10 s, a cui segue la quarantena. Nelle prove h1 non ha subito perdite durante la quarantena di h2 e il cambio di IP dalla stessa porta è stato contrastato. I limiti restano significativi (Sez. 6). Sviluppi futuri: rate limiting con *meter* OpenFlow (QoS) al posto del drop, detection di SYN/ICMP flood, binding da DHCP snooping, entropia del traffico per attacchi distribuiti low-rate.